anvl: Composable Code Transformations for Array Programs in R

by Sebastian Fischer, Daniel Falbel, Tomasz Kalinowski, Nikolai German, Martin Binder, Louis J. M. Aslett, Hugh Graham, and Bernd Bischl

Abstract We introduce the R package **anvl**, a code transformation framework for numerical array programs. Functions written in R against the package's array interface can be just-in-time (JIT) compiled, differentiated in reverse mode, and executed on CPUs and NVIDIA GPUs from the same source code. All transformations are based on tracing an R function into a computational graph, so arbitrary R code can be used to construct a program, while the program itself runs without the R interpreter. The main backend targets the OpenXLA compiler ecosystem via the companion packages **stablehlo** and **pjrt**, and an experimental second backend translates graphs to Fortran via **quickr**. A distinguishing feature is the treatment of data types: R values carry no data type of their own and are built into a program at the precision of the arrays they meet, and every function declares how it promotes its inputs, which gives users control over numerical precision without annotating the data they pass in from R. Except for the runtime, **anvl** and **stablehlo** are written in R, which makes them comparatively easy to extend with new primitives, transformations, and backends. We compare **anvl** with other R packages for compiled, differentiable, and GPU-accelerated computing, describe its design, and implement a Hamiltonian Monte Carlo sampler for Bayesian logistic regression, which we benchmark against Stan, **TMB**, and **torch** on CPUs and a GPU.

similar functional spirit as {anvl} * [x] stress that {quickr} is experimental and not really useable yet. (done: intro, abstract, tab:overview) * [x] mention interoperability: pjrt can run other {stablehlo} programs (done: Relation to JAX: pjrt runs programs exported from JAX) * [x] good benchmark comparing HMC from rstan with pure {anvl} implementation (also show GPU use). (done: Runtime evaluation) * [x] mention which bayesian frameworks are built on top of JAX in Python (done: intro: NumPyro, BlackJAX, TFP) * [x] mention that we test against torch for autodiff (done: Extending the package) * [x] future work: (done: Conclusion) * [x] jacobians (done: Conclusion) * [x] hessians (done: Conclusion) * [x] proper dynamism support with quickr backend (done: Conclusion) * [x] Stress that {anvl} is also about offering a more consistent API for array manipulating (to distinguish with {quickr}) (done: Distinguishing features, third point) * [x] Discuss eager and jit mode (done: Arrays and eager execution) * [x] Talk about async (done: Arrays and eager execution) * [x] CRAN submission: Download of prebuilt binaries (done: Background: R-universe instead of CRAN) * [x] include benchmark for tracing / compiler overhead and discuss: (done: Runtime: trace/lower/compile times) * [x] despite compiler overhead –> better scaling to expensive problems / large datasets (done: Runtime: compile times vs Stan/TMB) * [x] Create table that really does proper comparison with related work w.r.t. different features of the package (include in appendix maybe). (done: tab:comparison) * [x] compare with stan in more depth (AD engine; transpilation vs. proper XLA compiler) (done: tab:comparison, AD approach row) * [ ] include compile time in benchmark * [ ] use deep-research again for related work * [x] separating how from what (done: Introduction) * [x] anvl really low level infrastructure as a basis for others to develop new algorithms. (done: Introduction, user groups) * [x] static vs tracing: (done: Tracing R code into a graph) * [x] tracing requires control of the R function (now rowSums etc.) (done: Tracing: rowSums()) * [x] tracing allows to "borrow" more from the host interpreter (static arguments) (done: Tracing: static arguments)

Inline TODOs from the same file:

- ☒ cite the Autodidax tutorial and microjax (done: Autodidax in Acknowledgments, microjax in Package design)

☒ non-scalar (TODO) constants are not inlined into the StableHLO executable but added as program inputs – clarify which constants are inlined (done: Transforming and lowering graphs)
☒ HMC example (done: Use case: HMC)
☒ HMC benchmark (done: Runtime evaluation)
☒ Tracing vs. static analysis (done: Tracing R code into a graph) –>

1 Introduction

Most statistical and machine learning methods are, at their core, sequences of operations on arrays, which R can execute efficiently via vectorized operations and optimized linear algebra routines. When an algorithm is inherently iterative, as many optimization and sampling algorithms are, interpreted R loops become the bottleneck, and the usual remedy is to rewrite the performance-critical parts in C++ via **Rcpp** (**?**); if one additionally needs gradients or a GPU, the required machinery grows further.

In Python, these needs are covered by array libraries that combine hardware acceleration with automatic differentiation (AD), such as **PyTorch** (**?**) and **JAX** (**??**), on which much of the Bayesian inference ecosystem, including **NumPyro** (**?**), **BlackJAX** (**?**), and **TensorFlow Probability** (**?**), is built. The lack of comparable native infrastructure in R has consequences beyond runtime: methodological research on how to use accelerators for statistical methods can hardly be carried out in R today. For example, a recent line of work on Markov chain Monte Carlo (MCMC) runs hundreds or thousands of short chains in parallel on a GPU (**???**), and variational inference (**?**) turns posterior inference into a stochastic optimization problem (**??**). Both consist of many cheap, iterative gradient evaluations, which is where AD, compilation of the whole algorithm, and GPUs pay off most, and both are developed with JAX.

The core idea of **anvl**[1] is to bring JAX's approach of *composable code transformations* to R. Users write numerical algorithms in R against a set of array operations, most of which are available through the familiar R generics, and **anvl** transforms them into an executable via JIT compilation, into their gradient via reverse-mode AD, or both. The transformations are higher-order functions, similar in spirit to base R's `Vectorize()`, and they compose: the gradient of a function can be called from within a jitted function, which can in turn be called from another one. This separates what a program computes, written once in R, from how it is executed: on which device, at which precision, and whether compiled or differentiated. The primary backend targets the OpenXLA compiler ecosystem (**?**), which also powers JAX, via the companion packages **stablehlo** and **pjrt**, so the same program runs on CPUs and NVIDIA GPUs; an experimental second backend translates programs to Fortran via **quickr** (**?**). A focus of the design is control over numerical precision: R values have no data type until they meet an array, and every function declares how it promotes its inputs, so users can choose single or double precision per computation without annotating the data they pass in from R. Numerical functions, such as the density, distribution, and quantile functions, are implemented with algorithms chosen for accuracy at each precision, and the anvl-bench website[2] reports extensive accuracy benchmarks of the distribution functions against base R. Tables 1 and 2 compare **anvl** with other R packages for compiled and differentiable computing with and without GPU support, respectively, Table 3 lists its functionality in more detail, and Section 2.3 discusses its distinguishing features.

We see three main user groups: statisticians who implement iterative algorithms, such as samplers or optimizers, and want them to run fast without leaving R; methodological researchers who study algorithms that exploit accelerators, such as many-chain MCMC; and developers of R packages who need gradients and compiled code as infrastructure. For all of them, **anvl** is a low-level foundation rather than a modeling package, comparable to the role that JAX plays for the inference libraries built on it.

[1] https://github.com/r-xla/anvl
[2] https://r-xla.github.io/anvl-bench/

The remainder of this paper is structured as follows. Section 2 positions **anvl** among array libraries in other languages and related R packages. Section 3 introduces the OpenXLA compiler ecosystem. Section 4 describes the design of **anvl**, Section 5 its treatment of data types, and Section 6 how to extend it. In Section 7, we implement a Hamiltonian Monte Carlo sampler, whose runtime we evaluate in Section 8, and Section 9 concludes.

2 Related work

2.1 Array libraries and code transformation frameworks

The default language for numerical computing with accelerators is Python, with **TensorFlow** (**?**), **PyTorch** (**?**), and **JAX** (**?**). TensorFlow initially built a computational graph that was optimized as a whole, while PyTorch became popular for its eager execution, in which each operation runs immediately; today, TensorFlow also executes eagerly and PyTorch can compile programs via `torch.compile()` (**?**). JAX describes itself as a framework for composable transformations of NumPy programs (**?**): a function is traced into an intermediate representation, which can be differentiated in forward and reverse mode, vectorized via `vmap()`, and JIT compiled via the XLA compiler. Other languages have followed similar routes. In Julia (**?**), **Reactant.jl** (**?**) compiles Julia functions to StableHLO and runs them via XLA, **Nx** (**?**) traces numerical definitions in Elixir and compiles them with XLA, and **GoMLX** (**?**) does the same for Go via the PJRT wrapper **gopjrt** (**?**). The variety of these projects illustrates that OpenXLA's standardized interfaces (Section 3) allow a new language to obtain a compiler and hardware support without implementing them itself, which is the route that **anvl** takes for R.

2.2 Array computing in R

There are three routes to accelerated and differentiable array computing in R, whose representatives Tables 1 and 2 compare with **anvl**: wrapping a Python library via **reticulate** (**?**), using **torch**, or compiling a domain-specific or restricted language.

Wrappers of Python libraries. **tensorflow** (**?**), **keras3** (**?**), and **greta** (**?**), a probabilistic programming language built on TensorFlow Probability, give access to XLA-compiled programs, and JAX itself can be used via **reticulate**. They inherit the maturity and large feature set of these libraries, but also the costs of the language boundary. First, every session requires a Python environment with matching package versions, and as only one Python interpreter can be initialized per R session, two R packages with conflicting Python requirements cannot be used together (**?**). Second, values are converted between two type systems whose semantics differ: in **reticulate** 1.47.0, a numeric `NA` and `NaN` both become `nan`, an integer `NA` becomes the plain integer -2^{31}, and Python integers outside the 32-bit range can silently turn into wrong values when returned to R, e.g., `2**40` into `0` (**??**). Third, R's garbage collector does not know how much memory a Python object holds; in our experiments, repeatedly assigning an 800 MB NumPy array to the same R variable increased memory usage by about 760 MB per iteration until `gc()` was called. Finally, indices and reshaping follow Python's 0-based, row-major conventions unless a wrapper translates them function by function (**??**), and errors are reported as Python exceptions.

Native array libraries. **torch** (**?**) brings the **LibTorch** library that underpins PyTorch to R without a Python dependency and is the foundation of the deep learning ecosystem in R. Like PyTorch, it is designed for eager execution: tracing to TorchScript is possible, but TorchScript targets deployment and is no longer developed upstream, and `torch.compile()` operates on Python bytecode and is not available in R. For large neural networks, where the runtime is dominated by individually expensive operations, eager execution is rarely a bottleneck, but for the comparatively small models common in statistics, the per-operation overhead of dispatching each operation from R can dominate (Section 8). Packages that give R users direct access to GPUs have had a difficult history: CUDA packages such as **gputools** (**?**) were archived on CRAN between 2015 and 2018, **gpuR** (**?**), which provided

OpenCL-backed matrix classes, followed in November 2025, and **gpuMagic** (**?**), which compiled a subset of R to OpenCL, was removed from Bioconductor with release 3.23. Of the remaining packages, **OpenCL** (**?**) requires users to write kernels in OpenCL C, and **GPUmatrix** (**?**) builds on **torch** or **tensorflow**; this history is one reason why **anvl** delegates hardware support to OpenXLA's PJRT plugins.

Compilers and AD for R and domain-specific languages. The byte-code **compiler** included in R (**?**) compiles arbitrary R code, but because it has to preserve R's full semantics, the optimizations it can perform are limited (**?**). **quickr** (**?**) compiles a typed subset of R to Fortran, and **ast2ast** (**?**) translates a subset of R to C++ with AD, while **odin** (**?**) and **rxode2** (**?**) compile domain-specific languages for differential equations to C. Statistical modeling languages form another group. Stan (**?**) compiles its probabilistic programming language to C++ with a highly optimized reverse-mode AD library (**?**), is available from R via **rstan**, **cmdstanr** (**?**), and interfaces such as **brms** (**?**), and can evaluate some functions on GPUs via OpenCL. **nimble** (**?**) generates C++ from a BUGS-like language and from a subset of R, and **TMB** (**?**) differentiates C++ templates, both via CppAD (**?**); **RTMB** (**?**) records TMB's AD tape by evaluating R code with overloaded operators, and **dual** (**?**), **madness** (**?**), **nabla** (**?**), and **salad** (**?**) implement forward-mode AD via operator overloading in R. These packages are mature and highly optimized for the class of problems they target, but none of them combines JIT compilation of general R code, reverse-mode AD, and GPU execution.

2.3 Distinguishing features of anvl

To the best of our knowledge, **anvl** is the first native R package that offers JIT compilation of R code, reverse-mode AD, and GPU execution at the same time. Beyond this combination, it differs from the packages above in several ways. First, a program is *staged* in R: tracing evaluates the R function and records only the array operations, so, in contrast to **quickr** or **nimble**, any R code can be used to generate a program, while the program itself runs without the R interpreter and is optimized as a whole by XLA. Second, compilation and differentiation are transformations of R functions that compose, rather than properties of a separate model language as in Stan or **TMB**; a gradient can be used inside a compiled sampler that is itself called from another compiled function. Third, the array interface follows R rather than Python: indexing is 1-based, arrays are laid out in column-major order, and the common R generics work on arrays, whereas wrappers of Python libraries and, to a lesser extent, **torch** follow NumPy's conventions. In contrast to **quickr**, which compiles a subset of base R as it is, **anvl** provides an array library of its own that builds on the R generics but resolves inconsistencies of base R, such as the dropping of axes in subsetting (Section 4.2), and adds what base R lacks, such as further data types and random number generators with explicit state. Fourth, R values have no data type of their own and take the precision of the arrays they meet, and promotion rules are declared per function (Section 5), so single and double precision can be mixed deliberately, which matters on GPUs. Finally, the packages are extensible from R: primitives, AD rules, lowering rules, graph transformations, and backends are R objects (Section 6), and except for the PJRT runtime, **anvl** and **stablehlo** are pure R packages that need neither Python nor a local CUDA toolkit.

2.4 Relation to JAX

JAX, from which much of **anvl**'s design is adopted, provides forward-mode AD and, combined with reverse mode, efficient Jacobians and Hessians, whereas **anvl** computes gradients of scalar functions in reverse mode, which can be nested for higher-order derivatives, and cannot yet differentiate through the loop primitives `nv_scan()` and `nv_while()`. JAX's `vmap()` vectorizes a function automatically, while batched code, such as the multi-chain sampler in Section 7, has to be written by hand in **anvl**, and JAX can distribute a single program over many devices, whereas **anvl** places each array on one device. JAX also has a large ecosystem of libraries for optimization, probabilistic programming, and neural networks. On the other hand, **anvl** follows R's conventions, never converts R data to a default

	anvl	**torch**	**tensorflow**, **greta**	Stan (**cmdstanr**)
Program written in	R (array API)	R (array API)	R, run by Python libraries	Stan language
Program built by	tracing	– (eager); tracing (TorchScript)	tracing (`tf.function`)	parsing the model language
Execution	eager or compiled whole program	eager, per operation	eager or compiled graph	compiled model
Compiler	XLA; Fortran (experimental)	– (TorchScript tracing)	XLA (optional)	C++
AD	rev (scalar functions), HO (nested rev)	rev, HO	fwd, rev, HO	rev (fwd and HO internal only)
GPU	NVIDIA	NVIDIA, Apple	NVIDIA	(✓) (some functions, OpenCL)
Data types	bool, i8–i64, u8–u64, f32, f64 (chosen per computation)	bool, i8–i64, u8, f8, f16, bf16, f32, f64, complex	bool, i8–i64, u8–u64, f16, bf16, f32, f64, complex (**greta**: f32, f64)	i32, f64, complex
Data-dependent control flow	`nv_if()`, `nv_while()`, `nv_scan()`	R control flow	graph control flow	✓
Shapes	static, recompiled per shape	dynamic	static under XLA	dynamic
Conventions	1-based, column-major	1-based, row-major	0-based, row-major (partly translated)	1-based, column-major
New operations	primitives with rules in R; C/C++ custom calls	composition in R; C++	composition; Python, C++	C++ functions
Dependencies	PJRT plugin (downloaded)	LibTorch (downloaded)	Python, TensorFlow	C++ toolchain, CmdStan

Table 1: Comparison of **anvl** with R packages for compiled or differentiable numerical computing that support GPUs. Stan is used via **cmdstanr** or **rstan**. ✓: supported; (✓): possible with limitations; –: not available; fwd: forward mode; rev: reverse mode; HO: higher-order derivatives.

ISSN 2073-4859

	anvl	**quickr**	**TMB**, **RTMB**	**nimble**
Program written in	R (array API)	typed subset of R	C++ template; R (**RTMB**)	BUGS-like language, subset of R
Program built by	tracing	static analysis	C++ compilation (**TMB**); tracing (**RTMB**)	static analysis
Execution	eager or compiled whole program	compiled function	compiled C++, AD tape	compiled C++
Compiler	XLA; Fortran (experimental)	Fortran	C++	C++
AD	rev (scalar functions), HO (nested rev)	–	rev, HO (fwd internal only)	rev, HO (up to second order)
Data types	bool, i8–i64, u8–u64, f32, f64 (chosen per computation)	bool, i32, f64, complex	f64 (integers as data)	bool, i32, f64
Data-dependent control flow	`nv_if()`, `nv_while()`, `nv_scan()`	R control flow	✓	✓
Shapes	static, recompiled per shape	dynamic	dynamic	dynamic
Conventions	1-based, column-major	1-based, column-major	0-based in C++; 1-based in R	1-based, column-major
New operations	primitives with rules in R; C/C++ custom calls	–	atomic functions in C++	`nimbleFunction`, external C/C++
Dependencies	PJRT plugin (downloaded)	Fortran compiler	C++ toolchain (**TMB**)	C++ toolchain

Table 2: Comparison of **anvl** with R packages for compiled or differentiable numerical computing that run on the CPU only. ✓: supported; –: not available; fwd: forward mode; rev: reverse mode; HO: higher-order derivatives.

ISSN 2073-4859

Aspect	Functionality in **anvl**
Transformations	`jit()` with caching, static arguments, and buffer donation; `gradient()` and `value_and_gradient()` for reverse-mode AD of scalar functions, which can be nested; `with_dtypes()`; all of them compose.
Backends and devices	`"pjrt"` (XLA) on CPUs and NVIDIA GPUs via PJRT plugins, with several devices per platform; experimental `"quickr"` backend for 55 of the 101 primitives (f64, i32, and booleans). Asynchronous execution.
Data types	Booleans, signed and unsigned integers of 8 to 64 bits, f32, and f64. R values take the data type of the arrays they meet; scoped default data types and per-function promotion rules (Section 5).
Array interface	About 190 `nv_*()` functions and methods for about 100 R generics, including arithmetic, math, and comparison operators, reductions, `[` and `[<-`, `%*%`, `c()`, `cbind()`, `sort()`, `quantile()`, and cumulative functions; 1-based and column-major (Section 4.2).
Linear algebra	`solve()`, `chol()`, `qr()`, LU, SVD, symmetric eigendecomposition, and determinants via LAPACK (CPU) and cuSOLVER (GPU).
Random numbers	Generators with explicit state for normal, uniform, and binomial variates and sampling; density, distribution, and quantile functions of four distributions.
Control flow and I/O	`nv_if()`, `nv_while()`, and `nv_scan()`, and R control flow during tracing (Section 4.3); saving and loading arrays in the safetensors format.
Extensibility	New primitives with StableHLO, reverse-mode, and **quickr** rules via `new_primitive()`; custom calls into C/C++ via **pjrt**; new backends via `AnvlBackend` (Section 6).

Table 3: Overview of the functionality of **anvl**, complementing the comparison with other packages in Tables 1 and 2.

precision when it enters a program (Section 5), and makes the graph, its transformations, and the backends accessible as R objects; and as both lower to StableHLO, **pjrt** can also execute programs exported from JAX.

3 Background: the OpenXLA compiler ecosystem

OpenXLA (**?**) is an open-source compiler ecosystem for machine learning that originated in TensorFlow. It decouples array computing frameworks (the frontends) from hardware-specific compilers and runtimes (the backends) through two standardized interfaces: StableHLO (**?**), a portable intermediate representation for array programs, and PJRT (**?**), a runtime interface for compiling and executing such programs on a device. A framework that emits StableHLO and executes it via PJRT can run on any hardware for which a PJRT plugin exists, including CPUs, NVIDIA GPUs, and Google TPUs, which turns an $n \times m$ problem (n frontends, m types of hardware) into an $n + m$ problem. The main compiler consuming StableHLO is XLA, which, among other things, fuses operations to reduce memory traffic, but alternatives such as IREE (**?**) exist.

StableHLO is a dialect of MLIR, the Multi-Level Intermediate Representation of the LLVM project, and a strongly typed, functional language in static single assignment form, in which the type of every value, i.e., its data type and shape, is known when the program is compiled. Its around 100 operations range from elementwise arithmetic to control flow, reductions, and random number generation. The **stablehlo** package, which is written in R, provides an S3 representation of the language, type inference for its operations, and a builder API. Below, we create a program that multiplies two `f32` matrices elementwise; each `hlo_*()` call checks that the operation is valid for its inputs and records it in the function.

```
library(stablehlo)
func <- local_func("main")
x <- hlo_input("x", "f32", shape = c(2, 2))
y <- hlo_input("y", "f32", shape = c(2, 2))
func <- hlo_return(hlo_multiply(x, y))
func
```

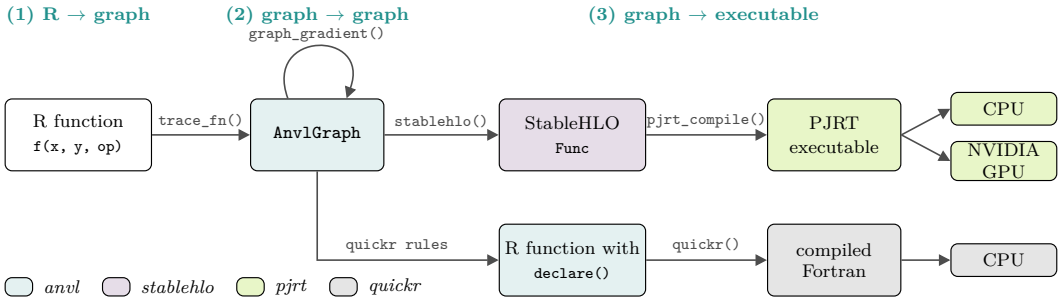


Figure 1: The three kinds of transformations in **anvl**. A node's color indicates the package that implements it. Users call `jit()`, `gradient()`, and `value_and_gradient()`, which apply these steps once the types of the inputs are known.

```
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = stablehlo.multiply %x, %y : tensor<2x2xf32>
#> return %0 : tensor<2x2xf32>
#> }
```

The **pjrt** package wraps the PJRT C API, which defines how to allocate buffers on a device, compile a program, and execute it; e.g., `pjrt_compile()` compiles the textual representation `repr(func)` of the program above, and `pjrt_execute()` runs it on buffers created by `pjrt_buffer()`.

As building a PJRT plugin requires compiling LLVM, MLIR, and XLA, **pjrt** downloads prebuilt plugins when they are first needed, after asking for permission, and a companion R package installs the NVIDIA libraries required by the CUDA plugin, so users need a compatible driver but no local CUDA toolkit. Because of these downloads, **anvl**, **stablehlo**, and **pjrt**, which are MIT-licensed, are currently distributed via R-universe rather than CRAN.

4 Package design

The central idea of **anvl**, adopted from JAX, is to view compilation and differentiation as code transformations, of which there are three kinds (Figure 1): an R function is traced into an `AnvlGraph`, a graph is transformed into another graph, e.g., by reverse-mode AD, and a graph is lowered to an executable by one of the backends. The minimalistic Python library microjax (**?**) gives a great summary of this code transformation approach.

4.1 Arrays and eager execution

The main data structure is the `AnvlArray`, which, compared to an R array, supports more data types, lives on a specific device, such as a CPU or a GPU, and represents scalars as arrays without axes. Operations are available through `nv_*()` functions and the corresponding R generics, and called on arrays, they are executed immediately, which is called *eager* execution. Plain R values take the data type of the array they meet, so the literal `2` below becomes an `f32` (Section 5).

```
library(anvl)
x <- nv_array(1:6, shape = c(2, 3), dtype = "f32")
sum(x * 2)
#> AnvlArray
#>  42
#> [ CPUf32{} ]
```

Under the hood, there are two layers of functions: *primitives*, named `prim_*()`, are the minimal operations that the transformations know about, each defined by a set of rules, and

the user-facing `nv_*()` functions build on them and add broadcasting of scalars, type promotion, and default arguments. Each primitive is itself a jitted function, so eager execution compiles and caches one small program per primitive and input type; this is convenient for interactive work, but for performance, the whole program should be compiled. Execution is asynchronous: calling a compiled program returns as soon as the computation has been scheduled, and its result is only awaited when its values are needed, e.g., by `as_array()`, which allows R to prepare the next call while the device is busy.

4.2 An R-like array interface

Array libraries such as NumPy, and with it JAX and PyTorch, follow Python's conventions: 0-based indexing, row-major memory layout, and broadcasting that aligns shapes from the last axis. **anvl** instead aims to feel familiar to R users, so that numerical code written for base R carries over with few changes. Arrays have methods for about 100 R generics (Table 3), whose semantics follow base R: indexing is 1-based, subsetting uses R's bracket syntax, and arrays are laid out in column-major order, which matters whenever an operation traverses the elements of an array. For example, `cumsum()` runs over a matrix column by column in both base R and **anvl**, whereas the same computation in **torch**, which requires flattening the tensor first, follows row-major order:

```
m <- matrix(1:6, nrow = 2)
cumsum(m)
#> [1]  1  3  6 10 15 21
as_array(cumsum(nv_array(m)))
#> [1]  1  3  6 10 15 21
t <- torch::torch_flatten(torch::torch_tensor(m))
torch::as_array(torch::torch_cumsum(t, dim = 1))
#> [1]  1  4  9 11 15 21
```

Where **anvl** needs functions of its own, their names and arguments follow base R, e.g., `nv_pmax()`, `nv_which_max()`, `nv_ifelse(test, yes, no)`, `nv_seq(from, to, by)`, and `nv_rnorm(shape, state, mean, sd)`. One deliberate departure concerns how R values correspond to arrays. A length-one R vector is a scalar, every other value must be an R array, whose `dim` becomes the shape, so each R value corresponds to exactly one shape. The same principle applies to subsetting: in base R, `m[1, ]` drops an axis unless `drop = FALSE` is given, so the type of the result changes when the number of selected rows happens to be one. In **anvl**, a scalar index drops the axis and an array index keeps it, whatever its length, which removes a discontinuity that would be particularly harmful in compiled programs, whose shapes are fixed:

```
x <- nv_array(matrix(1:6, 2))
shape(x[1, ])
#> [1] 3
shape(x[array(1), ])
#> [1] 1 3
```

The remaining deviations from base R follow from static shapes or from functional purity: only scalars are broadcast automatically, logical masks are not supported in subsetting, and random number generators take and return an explicit state.

4.3 Tracing R code into a graph

Instead of statically analyzing the source code of a function, as **quickr** does, **anvl** *traces* it: `trace_fn()` evaluates an R function with `GraphBox` objects in place of arrays, which carry only a data type and a shape. Below, we trace a function that adds or multiplies two `f32` scalars, created by `nv_aval()`, depending on an argument `op`.

```
f <- function(x, y, op) {
  if (op == "add") x + y else x * y
}
s <- nv_aval("f32", integer())
graph <- trace_fn(f, list(x = s, y = s, op = "mul"))
graph
#> <AnvlGraph> (%x1: f32[], %x2: f32[]) {
#>   %1: f32[] = mul(%x1, %x2)
#>   return %1
#> }
```

Every call to a primitive appends a `GraphStatement`, consisting of the primitive, its inputs, its static parameters, and its outputs, to the graph, while all other R code is simply evaluated; here, only the branch of the `if` statement that was taken becomes part of the `AnvlGraph`. The nesting structure of inputs and outputs is stored separately, so transformed functions can take and return (nested) lists of arrays.

Compared to static analysis, tracing allows arbitrary R code to construct a program and only needs to know the semantics of the array operations, not those of the R language as a whole. The downside is that tracing only works for functions that can be intercepted: `sum()` is a generic with a method for **anvl**'s arrays, whereas `rowSums()` is not generic and cannot be traced. Furthermore, programs must be pure: side effects, such as printing, only happen once during tracing, and R control flow can only depend on values that are known during tracing, such as static arguments and shapes.

4.4 Automatic differentiation

AD computes derivatives of a program by applying the chain rule to the elementary operations it executes, whose derivatives are known (**?**). *Forward mode* propagates derivatives from the inputs to the outputs alongside the evaluation, computing a Jacobian-vector product per pass, and is efficient for functions with few inputs. *Reverse mode* propagates the derivatives of the outputs with respect to each intermediate value, the *adjoints*, backwards to the inputs, computing a vector-Jacobian product per pass, and thus yields the gradient of a scalar function with many parameters, such as a log-likelihood, at a small constant multiple of the cost of evaluating it. Combining both, e.g., forward over reverse, gives Hessian-vector products and full Hessians efficiently. **anvl** currently implements reverse mode.

4.5 Transforming and lowering graphs

Once a program is represented as an `AnvlGraph`, it can be transformed into another graph. Currently, the main such transformation is reverse-mode AD, e.g., `graph_gradient(graph, wrt = c("x", "y"))`. For this, every primitive stores a `"reverse"` rule, which receives the inputs and outputs of the forward call and the adjoints of the outputs, and returns the vector-Jacobian products for the inputs. `graph_gradient()` walks the graph in reverse order, applies these rules, and accumulates the adjoints of values that are used multiple times. Because the reverse rules themselves call primitives, the backward pass is recorded with the same tracing machinery, and the result is again an `AnvlGraph`.

A backend then lowers a graph into an executable. For the `"pjrt"` backend, `stablehlo()` translates the graph into a StableHLO function by applying each primitive's `"stablehlo"` rule, which maps it to one or more **stablehlo** operations and converts R's 1-based indices to StableHLO's 0-based ones. R literals are embedded into the program as constants, while arrays that the traced function closes over become additional inputs, which avoids copying their data into the program. The result is compiled via **pjrt**. The **quickr** backend instead translates the graph into an R function with type declarations and compiles it with **quickr**; compared to XLA, a compiled **quickr** function can be reused for inputs of different shapes, but it only runs on the CPU and supports fewer data types.

4.6 The user interface

Users rarely call these transformations themselves, but use `jit()`, `gradient()`, and `value_and_gradient()`, which return functions that apply them lazily, once the types of the inputs are known. `jit()` returns a function that traces, lowers, and compiles its input when it is called, and caches the executable, so that a later call with inputs of the same data types and shapes reuses it. Below, the side effect shows that the second call does not trace the function again.

```
f_jit <- jit(function(x, y) {
  cat("tracing\n")
  x * y
})
y1 <- f_jit(x, x)
#> tracing
y2 <- f_jit(x, x + 1L)
```

Arguments that are not arrays, such as `op` above, must be marked as `static`, e.g., `jit(f, static = "op")`; their values become part of the cache key, and the function is traced again for every new value. A jitted function called from within another jitted function is traced as part of the outer function, so that both are compiled into a single executable. `gradient()` takes a function with scalar output and returns a function that computes the gradient with respect to the arguments given in `wrt`, which can itself be called within a jitted function (Section 7).

R's control flow can be used within jitted functions, but an `if` statement only retains the branch that was taken and a loop is unrolled during tracing. For control flow that depends on array values, or for loops that should not be unrolled, **anvl** provides the higher-order primitives `nv_if()`, `nv_while()`, and `nv_scan()`, which trace the functions passed to them into sub-graphs. For example, `nv_scan()` runs a loop for a fixed number of steps, threads a state through a body function, and stacks the outputs of all steps; we use it in Section 7.

Compiling whole programs also simplifies memory management on GPUs: in eager execution, every intermediate result is an R object that holds device memory until R's garbage collector reclaims it, which is why **torch** triggers the collector itself when device memory runs low (**?**), whereas in a jitted function, intermediate results are buffers that XLA allocates, reuses, and frees, and inputs can be donated via the `donate` option of `jit()`.

The main restriction stems from XLA's requirement that all shapes are known when a program is compiled. Operations whose output shape depends on data, such as `unique()`, cannot be expressed directly, and each new combination of input shapes triggers a new compilation; common workarounds are masking, e.g., via `nv_ifelse()`, and padding inputs to a few fixed sizes. Moreover, StableHLO programs cannot raise errors at runtime, so **anvl** performs all checks during tracing.

5 Data types and precision

Numerical precision is a recurring concern in statistics: log-likelihoods sum many terms, linear systems can be ill-conditioned, and MCMC methods compare densities whose difference is small relative to their size. Accelerators, on the other hand, are designed for single precision, and consumer GPUs often execute double-precision operations at a small fraction of the single-precision throughput. A framework for statistical computing should therefore make it easy to choose the precision of a computation, and it should not lose precision without being asked to.

5.1 R values have no data type

In JAX, Python scalars are *weakly typed*: they take the data type of the array they are combined with (**?**). NumPy arrays, however, have a data type, and by default, JAX truncates double-precision data to single precision when it enters a program, while with 64-bit mode enabled, the data promotes every single-precision array it meets. Transferred to R, where every real-valued vector is stored in double precision, either all data loses precision when it enters a program, or it pulls every computation into double precision.

anvl therefore extends weak typing to all R values that enter a program, i.e., length-one vectors as well as matrices and arrays. An R value has no data type and only *materializes* as an array when it meets one, taking its data type. Below, the same R matrix `B` is multiplied with a single- and a double-precision array.

```
B <- matrix(c(0.1, 0.2, 0.3, 0.4), 2, 2)
matmul <- jit(function(x, b) x %*% b)
c(dtype(matmul(nv_array(diag(2), dtype = "f32"), B)),
  dtype(matmul(nv_array(diag(2), dtype = "f64"), B)))
#> [1] "f32" "f64"
```

In the second call, `B` enters the program in double precision and the product is exactly `B`, whereas a fixed default precision would either have lost precision or turned the first result into `f64`. Only a value that meets no array, such as the argument of `nv_exp(1)`, settles on a default data type. A non-static R argument of a jitted function, such as `b` above, enters the trace with a shape but no data type, and each operation that uses it decides at which data type it materializes; if it is used at several data types, it is transferred at the narrowest one that holds all of them.

The same principle guards against silent changes of values elsewhere. An R value is built directly at the data type it takes, so a value that the data type cannot represent is an error: adding `-2L` to an unsigned 8-bit integer fails instead of wrapping around. As in R, `x - 1L` keeps an integer array `x` an integer, while `x - 1` gives a float. Finally, integers never become floats on their own: an unsigned and a signed 64-bit integer have no common data type and must be converted explicitly, whereas JAX and NumPy silently promote them to a float.

5.2 Choosing the precision

The default data types for R doubles and integers are a property of the backend (`f32` and `i32` for XLA) and can be changed for a scope via `with_default_dtypes()` or the option `anvl.default_dtypes`. `with_dtypes()` turns a function into one that runs at the given data types, converting its array arguments and results and setting the defaults for its body.

```
loss <- function(beta, X, y) mean((X %*% beta - y)^2)
loss_f64 <- with_dtypes(loss, c(float = "f64"))
c(dtype(nv_exp(1)), dtype(with_default_dtypes(c(float = "f64"), nv_exp(1))),
  dtype(loss_f64(nv_array(matrix(0, 2, 2), dtype = "f32"), B, B)))
#> [1] "f32" "f64" "f64"
```

Compiled programs are cached per default data types, so both variants can be used side by side, and in contrast to JAX's 64-bit mode, the defaults only decide what an R value settles on when nothing else determines its data type.

5.3 Promotion rules

When a function receives arrays of different data types, it has to decide on the data type of the computation. In **anvl**, this is not a global rule, but each `nv_*()` function passes a *promotion rule* to `as_anvl_arrays()`, which converts its arguments to arrays at the beginning

of the function. `promotion_common()` brings all inputs to their common data type, as used, e.g., by `nv_add()`; `promotion_like()` brings them to the data type of one designated input; `promotion_dtype()` to a fixed data type; and `promotion_rdata_common()`, which is used by primitives, only lets R values adapt and requires arrays to agree. The rules refuse conversions that lose precision unless `coerce = TRUE`. For example, the following function computes $ax + y$ at the data type of `y`, and refuses an `x` of higher precision rather than silently rounding it.

```
axpy <- jit(function(a, x, y) {
  args <- as_anvl_arrays(a = a, x = x, y = y, .promote = promotion_like("y"))
  args$a * args$x + args$y
})
y32 <- nv_array(c(1, 1), dtype = "f32")
as_array(axpy(2, nv_array(c(1, 2), dtype = "f32"), y32))
#> [1] 3 5
axpy(2, nv_array(c(1, 2), dtype = "f64"), y32)
#> Error:
#> ! Cannot bring `x` to data type "f32".
#> x "f64" is not promotable to "f32".
#> i Convert it explicitly with `nv_convert()`.
```

Developers of packages built on **anvl** can thus decide for each function which argument determines the precision, e.g., that a likelihood is accumulated in double precision while the data stays in single precision.

6 Extending the package

Most operations can be expressed in terms of existing primitives and are best added as `nv_*()` functions. A new primitive is needed if an operation cannot be expressed otherwise, or if it benefits from a dedicated implementation, e.g., of its derivative. Below, we add a primitive for the softplus function $\log(1 + e^x)$, which appears in the logistic regression example in Section 7. A primitive is created via `new_primitive()`, whose body records a statement in the graph via `graph_desc_add()`, together with an inference function that computes the types of the outputs from those of the inputs and validates them. Within the body, `self` refers to the primitive being defined.

```
prim_softplus <- new_primitive("softplus", function(x) {
  infer <- function(x) {
    if (!(dtype(x) %in% c("f32", "f64"))) stop("`x` must be a float.")
    list(AbstractArray(dtype = dtype(x), shape = shape(x)))
  }
  graph_desc_add(self, list(x = x), infer_fn = infer)[[1L]]
})
```

Next, we attach the StableHLO rule, which receives and returns **stablehlo** values, and a reverse rule, which uses the derivative $\sigma(x) = 1/(1 + e^{-x})$.

```
prim_softplus[["stablehlo"]] <- function(x) {
  list(hlo_log_plus_one(hlo_exponential(x)))
}
prim_softplus[["reverse"]] <- rule_reverse(
  function(inputs, outputs, grads, params, required) {
    list(prim_mul(grads[[1L]], prim_plogis(inputs[[1L]])))
  }
)
```

The primitive can now be used eagerly, within jitted functions, and be differentiated.

```
grad_softplus <- jit(gradient(function(z) sum(prim_softplus(z))))
as_array(grad_softplus(nv_array(c(-1, 0, 1)))$z)
#> [1] 0.2689414 0.5000000 0.7310586
```

Operations that cannot be expressed in StableHLO can use PJRT's custom call mechanism: a handler written in C or C++ against XLA's foreign function interface is registered from R via **pjrt**, with separate implementations per platform, as the linear algebra primitives do to call LAPACK and cuSOLVER.

Since an `AnvlGraph` is a simple data structure, new graph-to-graph transformations, such as forward-mode AD, can be implemented in R by iterating over its statements and emitting new ones, guided by a rule stored in each primitive. A backend is an `AnvlBackend` object, a collection of functions that define how arrays are stored, converted to R, and placed on devices, and how a graph is compiled, together with a lowering rule for each supported primitive; exactly one backend is active at a time. We test every primitive against **torch** as a reference implementation wherever possible, comparing the outputs and gradients of jitted primitives with those of **torch** and its autograd engine on random inputs.

7 Use case: Hamiltonian Monte Carlo

Hamiltonian Monte Carlo (HMC) (**?**) is an MCMC method that uses the gradient of the log posterior density to propose distant states that are still accepted with high probability, and thus a natural use case for **anvl**: it needs gradients, its loops are slow in R, and running many chains in parallel is a good fit for GPUs. We consider Bayesian logistic regression with a binary response $y \in \{0,1\}^n$, features $X \in \mathbb{R}^{n \times p}$, $y_i \sim \text{Bernoulli}(\sigma(x_i^\top \beta))$, and a standard normal prior $\beta \sim \mathcal{N}(0, I_p)$. HMC simulates the dynamics of a particle with position β and momentum $\rho \sim \mathcal{N}(0, I_p)$ in the potential energy

$$U(\beta) = -\log p(\beta \mid X, y) = -\sum_{i=1}^{n} \left(y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta}) \right) + \frac{1}{2}\|\beta\|^2 + \text{const.}$$

We write the sampler for K chains at once, so the positions are stored in a $K \times p$ matrix, and the potential returns a vector of length K.

```
potential <- function(beta, X, y) {
  eta <- X %*% t(beta)
  y <- nv_broadcast_to(y, shape(eta))
  loglik <- nv_sum(y * eta - nv_log1p(exp(eta)), axes = 1L)
  logprior <- -nv_sum(beta^2, axes = 2L) / 2
  -(loglik + logprior)
}
```

Because the chains are independent, the gradient of the summed potential contains the gradient of each chain in the corresponding row, so a single call to `gradient()` yields the gradients of all chains.

```
grad_potential <- gradient(
  function(beta, X, y) sum(potential(beta, X, y)),
  wrt = "beta"
)
```

The dynamics are simulated with the leapfrog integrator, which alternates half steps of size $\varepsilon/2$ for the momentum with full steps of size ε for the position. We run L such steps with `nv_scan()`, where the gradient at the current position is part of the carry, so that each step evaluates the gradient only once.

```
leapfrog <- function(beta, p, X, y, eps, L) {
  init <- list(beta = beta, p = p, grad = grad_potential(beta, X, y)$beta)
  nv_scan(init, steps = L, body = function(s, x) {
    p <- s$p - eps / 2 * s$grad
    beta <- s$beta + eps * p
    grad <- grad_potential(beta, X, y)$beta
    p <- p - eps / 2 * grad
    list(carry = list(beta = beta, p = p, grad = grad), out = NULL)
  })$carry
}
```

A transition draws a new momentum, simulates a trajectory, and accepts the proposal of each chain with probability $\min(1, \exp(H_0 - H_1))$, where $H = U(\beta) + \|\rho\|^2/2$ is the total energy. The random number generator state is passed explicitly, and the acceptance decision uses `nv_ifelse()` instead of an R `if` statement, as it depends on array values.

```
hmc_step <- function(beta, state, X, y, eps, L) {
  draw <- nv_rnorm(shape(beta), state)
  p <- draw$values
  prop <- leapfrog(beta, p, X, y, eps, L)
  H0 <- potential(beta, X, y) + nv_sum(p^2, axes = 2L) / 2
  H1 <- potential(prop$beta, X, y) + nv_sum(prop$p^2, axes = 2L) / 2
  u <- nv_runif(shape(beta)[1L], draw$state)
  accept <- log(u$values) < H0 - H1
  accept <- nv_broadcast_to(accept, shape(beta))
  list(beta = nv_ifelse(accept, prop$beta, beta), state = u$state)
}
```

Finally, we collect `n_draws` draws with another `nv_scan()`, whose outputs are stacked along a new first axis, and compile the whole sampler with `jit()`. The number of leapfrog steps and draws are static arguments, as they determine the structure of the program.

```
hmc <- jit(function(beta, state, X, y, eps, L, n_draws) {
  out <- nv_scan(list(beta = beta, state = state), steps = n_draws,
    body = function(s, x) {
      s <- hmc_step(s$beta, s$state, X, y, eps, L)
      list(carry = s, out = s$beta)
    }
  )
  list(draws = out$out, beta = out$carry$beta, state = out$carry$state)
}, static = c("L", "n_draws"))
```

We apply the sampler to simulated data with $n = 1000$ observations and $p = 5$ features, running four chains in double precision. The posterior means, computed from the second half of each chain, agree with the maximum likelihood estimates up to the shrinkage induced by the prior.

```
options(anvl.default_dtypes = c(float = "f64", int = "i32"))
set.seed(1)
n <- 1000; p <- 5
X <- matrix(rnorm(n * p), n, p)
y <- rbinom(n, 1, plogis(X %*% rnorm(p, sd = 0.5)))
res <- hmc(nv_array(matrix(0, 4, p)), nv_rng_state(42L), nv_array(X),
  nv_array(as.double(y)), eps = 0.5 / sqrt(n), L = 10L, n_draws = 2000L)
draws <- as_array(res$draws)
rbind(hmc = colMeans(apply(draws[1001:2000, , ], 3, c)),
```

```
  glm = coef(glm(y ~ X - 1, family = binomial)))
#>             X1        X2         X3        X4        X5
#> hmc -0.7730560 0.3412134 -0.8296505 0.6687511 0.5593170
#> glm -0.7738017 0.3425455 -0.8321399 0.6701697 0.5599573
```

The whole sampler, including the 2000×11 gradient evaluations per chain, runs as a single compiled program without returning to R. The same code runs on a GPU when the arrays are allocated there, e.g., via `nv_array(X, device = "cuda")`.

8 Runtime evaluation

We compare the runtime of the HMC sampler from Section 7 to Stan, whose AD library and code generation are the state of the art for statistical models on the CPU (**?**), to **TMB**, and to the same sampler written with **torch**.

8.1 Setup

We run Stan's static HMC sampler (without the No-U-Turn extension) via CmdStan 2.40 with the same step size $\varepsilon = 0.5/\sqrt{n}$, $L = 10$ leapfrog steps, the identity metric, and adaptation disabled, so that all samplers perform the same number of gradient evaluations. The Stan model uses `bernoulli_logit_glm()`, Stan's most efficient implementation of the logistic regression likelihood, which can also be evaluated on a GPU via OpenCL; on the GPU, Stan's chains run as separate processes that share the device, at most 8 at a time. **TMB** evaluates the log posterior and its gradient with CppAD from a C++ template, and as it does not provide an HMC sampler, we run the same static HMC loop in R around its compiled objective and gradient, as is common when custom algorithms are implemented with **TMB**. The **torch** sampler mirrors the **anvl** code, but runs the leapfrog and sampling loops in R, as **torch** executes each operation eagerly. All samplers work in double precision and start at $\beta = 0$, and the data are simulated with standard normal features and $\beta \sim \mathcal{N}(0, 0.25 I_p)$. We exclude process start-up and data loading from Stan's measurements by subtracting the time of an otherwise identical run without draws, and time **anvl** after the data has been transferred to the device. Each configuration is repeated three times after a first call that includes compilation, and we report the median.

We run three experiments. First, we draw 1000 samples from a single chain for $p = 20$ and $n \in \{10^3, 10^4, 10^5\}$. Second, we measure strong scaling: we draw 200 samples from a single chain for $n = 10^5$ and $p = 100$ on 1 to 16 cores. **anvl** and **torch** run the unchanged sampler, which they parallelize within operations, **TMB** evaluates the likelihood with several OpenMP threads, and Stan needs a modified model, in which `reduce_sum()` splits the likelihood into slices that are evaluated by several threads. Third, we draw 1000 samples from each of K chains for $n = 10^4$ and $p = 20$. CmdStan and **TMB** run one chain per process, at most one per core, and **anvl** runs all chains as one batched program, except on 16 cores, where we split them into 16 groups that run on separate PJRT CPU devices. All experiments ran on the LRZ AI Systems, on two Intel Xeon Gold 6336Y processors restricted to 16 physical cores and on an NVIDIA A100 80GB GPU, in a Docker image that pins all software versions; the image, the benchmark scripts, and instructions are in the `paper/` directory of `https://github.com/r-xla/anvl`.

8.2 Results

Figure 2 (left) shows the results for a single chain. On one core, **anvl** draws the samples 3.1, 2.7, and 1.7 times faster than CmdStan for $n = 10^3$, 10^4, and 10^5, respectively. A plausible reason is that XLA compiles the whole sampler into a single optimized program, whereas Stan and **TMB**, which takes 11 to 13 times as long as CmdStan, evaluate the gradient with operator-overloading AD. The GPU does not pay off for small data, but for $n = 10^5$, it draws the samples in 0.31 seconds, about 90 times faster than **anvl** on one core and 150 times faster

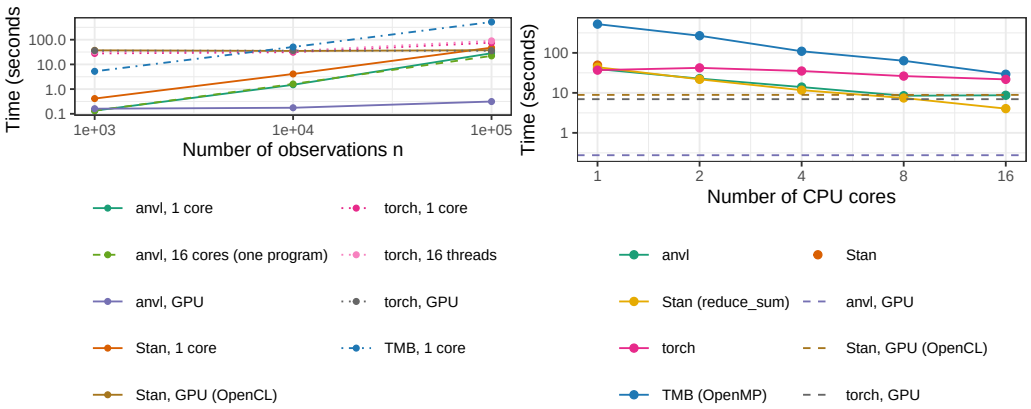


Figure 2: Left: time to draw 1000 samples from a single chain as a function of the number of observations n ($p = 20$). Right: strong scaling, time to draw 200 samples from a single chain for $n = 10^5$ and $p = 100$ on 1 to 16 CPU cores; Stan uses reduce_sum(), the single point is the unmodified Stan model, and the dashed lines are **anvl**, **torch**, and Stan (OpenCL) on the GPU. Medians of three repetitions, log-log scales.

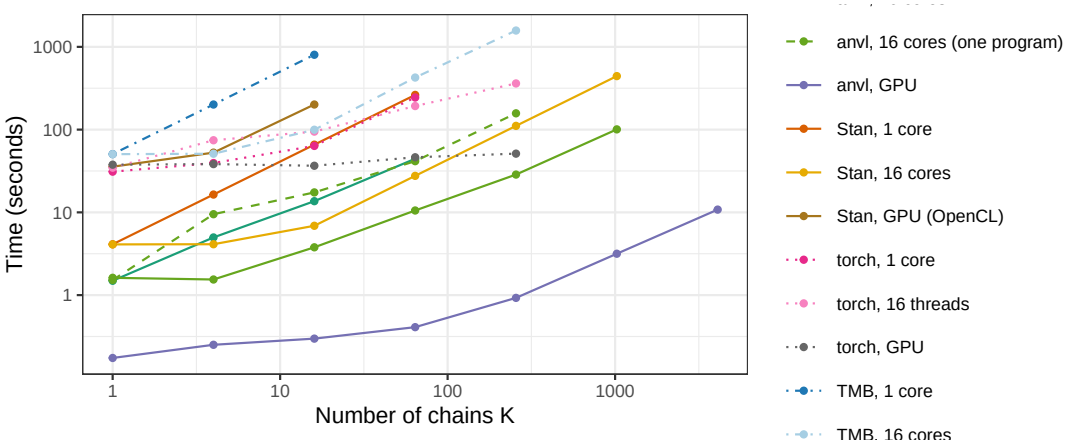


Figure 3: Time to draw 1000 samples from each of K chains for $n = 10^4$ (median of three repetitions, log-log scale).

than CmdStan. The **torch** sampler needs about 28 seconds for $n = 10^3$ on one core, 200 times as long as **anvl**, and about 36 seconds for every n on the GPU: its runtime is dominated by the overhead of dispatching each operation from R, which is the per-operation overhead of eager execution discussed in Section 2.2. Stan's OpenCL backend, which evaluates only the likelihood and its gradient on the GPU, needs 35 to 38 seconds for every n, as each gradient evaluation transfers data and launches kernels.

Figure 2 (right) shows how a single chain scales with the number of cores. Without any change to the code, **anvl** becomes 1.7, 2.8, and 4.6 times faster on 2, 4, and 8 cores, and does not improve further on 16 cores, where the individual operations are too small at this problem size to keep all cores busy. With `reduce_sum()`, Stan scales further (11 times faster on 16 threads), but this requires rewriting the model, which is then slower on a single core (44 versus 50 seconds, compared to 39 seconds for **anvl**). **TMB** becomes 18 times faster on 16 threads, but starts from 522 seconds, and **torch** is as fast as **anvl** on one core, where the operations are large enough to hide its overhead, but becomes only 1.7 times faster on 16 threads. On the GPU, the same run takes 0.28 seconds with **anvl**, 7 seconds with **torch**, and 8.9 seconds with Stan's OpenCL backend.

Figure 3 shows the results for multiple chains. On one core, **anvl** is 2.8 (one chain) to 5.9 (64 chains) times faster than CmdStan, as batching turns the matrix-vector products of the individual chains into matrix-matrix products. **torch** benefits from batching as well, but is still 5.5 times slower than **anvl** for 64 chains on one core, and **TMB** takes 55 to 59

times as long as **anvl**. On 16 cores, **anvl** is 2.6 times faster than CmdStan for 64 chains and 4.4 times faster for 1024 chains; running all chains as a single program instead of splitting them over devices is considerably slower, because each operation is again too small to be distributed efficiently. On the GPU, the time of **anvl** is almost constant up to 64 chains and grows sublinearly beyond: 4096 chains with 1000 draws each take 10.8 seconds, about 380,000 draws per second, compared to 2,300 for CmdStan on 16 cores. This is the regime in which the many-chain MCMC methods discussed in the introduction operate; on the GPU, **torch** did not finish 1024 chains and Stan's OpenCL backend did not finish 64 chains within one hour. All average acceptance rates were between 0.97 and 0.99.

Finally, the one-off costs are small: for $n = 10^4$ on 16 cores, tracing the sampler, lowering it to StableHLO, and compiling it with XLA take 0.28, 0.21, and 0.24 seconds, respectively, compared to about 11 seconds for compiling the Stan model and 35 seconds for the **TMB** template.

9 Conclusion

We introduced **anvl**, which brings composable code transformations for array programs to R: programs are traced into a graph, which can be differentiated in reverse mode and compiled via OpenXLA or **quickr**, so the same R code runs on CPUs and GPUs, with an R-like array interface and control over numerical precision. The HMC use case showed that a whole sampler can be compiled into a single program that is faster than Stan on the CPU and scales to thousands of chains on a GPU.

The main limitations stem from XLA's static shapes, which rule out data-dependent output shapes and require recompilation for new shapes, and from AD, which is restricted to reverse mode and cannot yet differentiate through loops. Future work will add forward-mode AD, which, combined with reverse mode, yields Jacobians and Hessians efficiently, e.g., for Laplace approximations, and a vectorizing transformation similar to JAX's `vmap()`, which would remove the need to write batched code by hand. The **quickr** backend and the IREE compiler (**?**), which also consumes StableHLO, are two routes towards dynamic shapes. Beyond the core, we hope that **anvl** will serve as a foundation for packages that implement statistical and machine learning algorithms, such as samplers and variational inference, on top of it. A first such package is **garry** (**?**), a lazy raster engine for remote sensing that compiles the numerical kernels of a raster computation with **anvl** and executes them chunked across processes.

Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 460135501 (NFDI project MaRDI). We thank the developers of JAX, whose design and documentation, in particular the Autodidax tutorial (**?**), inspired many aspects of **anvl**, as well as the authors of the Go library gopjrt (**?**), which informed the design of **pjrt**. We also thank Doug Kelkhoff, the main author of an experimental implementation of the R language (`https://github.com/dgkf/R`), which sparked parts of the design of **anvl**. We gratefully acknowledge the computational resources provided by the Leibniz Supercomputing Centre (LRZ). AI coding agents were used in the development of the software and the preparation of the manuscript; all AI-generated content was reviewed by the authors.

Sebastian Fischer
LMU MunichMunich Center for Machine Learning (MCML)
Department of Statistics
Ludwig-Maximilians-Universität München
Ludwigstr. 33, 80539 Munich, Germany

ORCiD: 0000-0002-9609-3197
sebastian.fischer@stat.uni-muenchen.de

Daniel Falbel
Posit Software, PBC

ORCiD: 0009-0006-0143-2392

Tomasz Kalinowski
Posit Software, PBC

Nikolai German
LMU Munich

ORCiD: 0009-0001-7394-8367

Martin Binder
LMU MunichMunich Center for Machine Learning (MCML)

ORCiD: 0009-0008-2578-2869

Louis J. M. Aslett
Durham University
Department of Mathematical Sciences
ORCiD: 0000-0003-2211-233X

Hugh Graham
Belian

ORCiD: 0000-0001-9451-5010

Bernd Bischl
LMU MunichMunich Center for Machine Learning (MCML)

ORCiD: 0000-0001-6002-6980